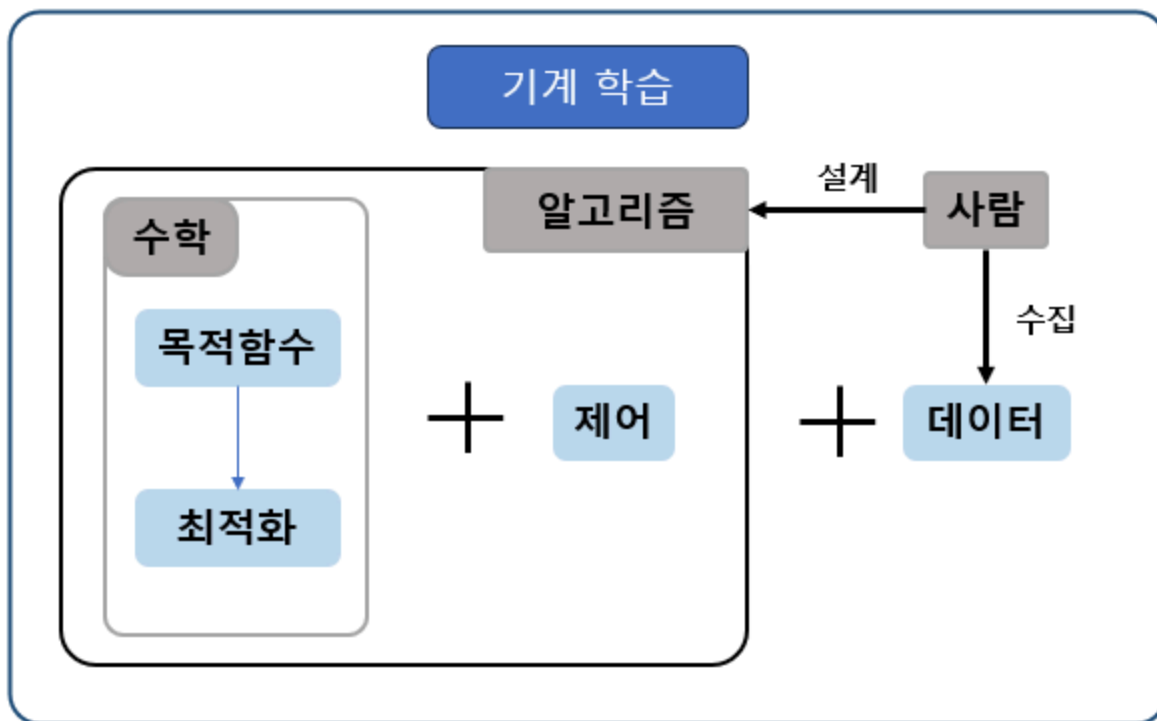


머신러닝 교재 2

기계 학습에서 수학의 역할

수학은 목적 함수를 정의하고, 목적 함수가 최저가 되는 점을 찾아주는 최적화 이론을 제공한다. 최적화 이론에 규제, 모멘텀, 학습률, 멈춤(stop)조건과 같은 제어를 추가하여 알고리즘 구축한다. 사람을 해당 알고리즘을 설계하고 데이터를 수집한다.



벡터

샘플을 특징 벡터로 표현한다.

- ex): Iris 데이터에서 꽃 받침의 길이, 꽃 받침의 너비, 꽃잎의 길이, 꽃잎의 너비라는 4개의 특징이 각각 5.1, 3.5, 1.4, 0.2인 샘플

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{pmatrix}$$

- 여러 개의 특징 벡터를 첨자로 구분한다.

$$\mathbf{x}_1 = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 4.9 \\ 3.0 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_3 = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 4.7 \\ 3.2 \\ 1.3 \\ 0.2 \end{pmatrix}, \dots, \mathbf{x}_{150} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 5.9 \\ 3.0 \\ 5.1 \\ 1.8 \end{pmatrix},$$

행렬

행렬(matrix)은 여러 개의 숫자(스칼라)를 **행(row)과 열(column)의 형태로 정렬한 2차원의 구조**이며, 기계학습에서 데이터를 수학적으로 표현하고 연산하기 위해 필수적인 도구이다.

특히, **여러 개의 샘플을 특징 벡터(feature vector)로 표현한 후 이들을 하나로 모아 구성한 것이 바로 행렬**이다. 예를 들어, Iris 데이터셋처럼 각 샘플이 2개의 특징(꽃받침의 길이, 너비, 꽃잎의 길이, 너비)을 가진 경우, 150개의 샘플은 150x4 크기의 **설계 행렬(design matrix)**로 표현될 수 있다.

- ex) Iris 데이터에 있는 150개의 샘플을 설계 행렬 X로 표현

$$\mathbf{X} = \begin{pmatrix} 5.1 & 3.5 & 1.4 & 0.2 \\ 4.9 & 3.0 & 1.4 & 0.2 \\ 4.7 & 3.2 & 1.3 & 0.2 \\ 4.6 & 3.1 & 1.5 & 0.2 \\ \vdots & \vdots & \vdots & \vdots \\ 6.2 & 3.4 & 5.4 & 2.3 \\ 5.9 & 3.0 & 5.1 & 1.8 \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ x_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{149,1} & x_{149,2} & x_{149,3} & x_{149,4} \\ x_{150,1} & x_{150,2} & x_{150,3} & x_{150,4} \end{pmatrix}$$

- 행(row): 하나의 데이터 샘플
- 열(column): 하나의 특성(feature)

행렬을 사용하면 **복잡한 연산을 간결한 수식**으로 표현할 수 있으며, **벡터의 내적, 행렬 곱, 전치(transpose)** 같은 기본 연산을 통해 다양한 알고리즘의 수학적 기초를 구성할 수 있다. 특히 **다변수 선형회귀, 신경망의 순전파/역전파, PCA 같은 축소 기법**에서도 행렬 연산이 핵심 역할을 한다.

- ex) 다항식의 행렬 표현

$$\begin{aligned} f(\mathbf{x}) &= f(x_1, x_2, x_3) \\ &= 2x_1x_1 - 4x_1x_2 + 3x_1x_3 + x_2x_1 + 2x_2x_2 + 6x_2x_3 - 2x_3x_1 + 3x_3x_2 + 2x_3x_3 \\ &\quad + 2x_1 + 3x_2 - 4x_3 + 5 \\ &= (x_1 \quad x_2 \quad x_3) \begin{pmatrix} 2 & -4 & 3 \\ 1 & 2 & 6 \\ -2 & 3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + (2 \quad 3 \quad -4) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + 5 \\ &= \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c \end{aligned}$$

</>

또한, 행렬은 **벡터의 집합**이라는 의미를 갖기 때문에 **고차원 벡터 공간**을 구성하거나, **선형 결합**을 통해 새로운 벡터를 생성하는 데 활용된다. 이처럼 행렬은 단순한 숫자 모음이 아니라, 데이터의 구조와 연산의 규칙을 수학적 표현하는 수단으로, 기계학습과 딥러닝의 모든 알고리즘에 필수적으로 사용된다.

Numpy

numpy는 다차원 배열을 효율적으로 처리하고 벡터화 연산, 선형 대수 생성, 푸리에 변환 등 다양한 수학/과학 계산에 주로 사용되는 고성능 수치 계산을 위한 라이브러리다.

import 구문을 통해 코드 밖에 있는 파이썬 파일의 함수와 클래스를 사용할 수 있다.

- numpy를 불러올 때는 주로 **as** 를 사용하여 **np**로 줄여 사용한다.

```
import numpy as np
```

PYTHON

필요성

- C/C++과 같은 저수준 언어 기반의 호환 API 제공
- 수행 성능이 매우 중요한 부분은 C/C++ 기반으로 코드로 작성 -> 이를 numpy에서 호출하면 통합 가능
 - 구글의 딥러닝 프레임워크인 **tensorflow**가 위의 방식 사용, 배열 연산 수행 속도를 개선, **numpy**와 호환될 수 있게 작성되었음
- 많은 머신러닝 알고리즘이 **넘파이 기반으로 작성됨**
- 또한 ML 알고리즘의 입/출력 데이터가 넘파이 배열 타입으로 사용되므로 이해할 필요가 있음

Python List vs NumPy Array

- Python List
 - 여러가지 타입의 원소를 저장 가능
 - 메모리 용량이 크고 속도가 느림
 - 전체 연산 불가
- NumPy Array
 - 같은 타입의 원소만 저장
 - 메모리 최적화, 계산 속도 향상
 - 크기(dimension)이 명확하게 정의

NumPy 대표 함수들

1) np.array

`array`는 `numpy`의 가장 기본적인 연산 단위인 배열 클래스다.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])

print(a)
print(b)

print(a.shape)
print(b.shape)

print(a.dtype)
print(b.dtype)

print(a.ndim)
print(b.ndim)
```

PYTHON

`array` 인스턴스 선언 시 다차원의 리스트를 인자로 사용하여 선언 가능하다. 해당 클래스의 기본적인 변수는 `shape`, `dtype`, `ndim` 등이 있다.

- `shape`: 배열 크기를 튜플로 저장
- `dtype`: 배열 요소의 자료형을 저장. `array`의 모든 요소는 같은 자료형으로 통일 됨.
- `ndim`: 배열의 원 수를 저장.

2) 배열을 원하는 대로 생성하는 함수들

```
print(np.zeros((2, 3)))
print(np.ones((3, 3)))

print(np.eye(4))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))

print(np.full(2, 2), 7)
```

PYTHON

- `zeros`: 인자로 받은 크기만큼 0 요소로 찬 배열을 생성
- `ones`: 인자로 받은 크기만큼 1 요소로 찬 배열을 생성
- `eye`: 인자로 받은 크기만큼 단위 행렬 생성

- `arange`: 등차 수열의 정수 배열을 생성
- `linspace`: 등차 간격으로 나눈 실수 배열 생성
- `full`: 배열에 사용자가 지정한 값을 삽입

3) `np.reshape()`

`np.reshape()`은 내부 데이터는 변경하지 않으면서 배열의 구조를 바꾼다.

```
reshape_mat = np.array(np.arange(30)).reshape((5,6))
print(reshape_mat)
```

PYTHON

- 0부터 29까지의 숫자를 생성하는 `arrange(30)`을 수행한 후, 원소의 개수가 30개이므로 5행 6열의 행렬로 변경된다.

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]
 [24 25 26 27 28 29]]
```

4) `np.random.random()`

`np.random.random()`은 임의의 값을 가지는 배열을 생성한다.

```
random_mat = np.random.random((2,2)) # 임의의 값으로 채워진 배열 생성
print(random_mat)
```

PYTHON

```
[[0.3111881  0.72996102]
 [0.65667734 0.40758328]]
```

행렬 연산

- 2x2의 A행렬과, B 행렬을 각각 정의하고 출력하라.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("행렬 A:\n", A)
print("행렬 B:\n", B)
```

C 행렬은 A 행렬과 B 행렬의 덧셈, D 행렬은 A에서 B를 뺀 행렬, E 행렬은 B에서 A를 뺀 행렬을 나타내는 행렬이다. 행렬의 덧셈 및 뺄셈의 대해 행렬을 정의하고, 출력하라.

```
C = A + B
D = A - B
E = B - A

print("A + B:\n", C)
print("A - B:\n", D)
print("B - A:\n", E)

print("A @ B:\n", A @ B)
print("B @ A:\n", B @ A)
```

위 결과에서 D와 E는 서로 다른 값을 가지므로, **행렬의 뺄셈은 교환 법칙이 성립하지 않는다**는 것을 확인할 수 있다. 또한, 행렬 곱셈 역시 일반적으로 교환 법칙이 성립하지 않는다.

$$A @ B \neq B @ A$$

- 행렬 곱셈은 일반적으로 교환법칙이 성립하지 않는다.

$$A @ B \neq B @ A$$

- 2x2 행렬 **x**, **y**, **z**를 정의해주고, 분배 법칙이 성립하는지 확인해보자.

$$A(B + C) = AB + AC$$

```

X = np.array([[1, 2], [3, 4]])
Y = np.array([[2, 0], [1, 2]])
Z = np.array([[0, 1], [3, 1]])

left = X @ (Y + Z)

right = X @ Y + X @ Z

print("X @ (Y + Z):\n", left)
print("X @ Y + X @ Z:\n", right)
print("동일한가?:", np.allclose(left, right))

```

위에서 정의한 **x**, **y**, **z** 행렬을 그대로 사용하여, 결합법칙

$$A(BC) = (AB)C$$

이 성립하는지 확인해보자.

```

left = X @ (Y @ Z)

right = (X @ Y) @ Z

print("X @ (Y @ Z):\n", left)
print("(X @ Y) @ Z:\n", right)
print("동일한가?:", np.allclose(left, right))

```

행렬 연산

- 행렬 곱셈에서 교환법칙은 성립하지 않는다:

$$A@B \neq B@A$$

- 분배 법칙과 결합 법칙이 성립한다:

$$A(B + C) = AB + AC$$

$$A(BC) = (AB)C$$

행렬 A의 전치 행렬 A^T

전치행렬이란, 행렬의 행(row)과 열(column)을 뒤바꾼 것이다.

수학적으로는 **A^T** 또는 **A.T**로 표현한다.

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix}, \quad A^T = \begin{pmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1m} & a_{2m} & \cdots & a_{nm} \end{pmatrix}$$

예를 들어,

$$A = \begin{pmatrix} 3 & 4 & 1 \\ 0 & 5 & 2 \end{pmatrix}$$

라면

$$A^T = \begin{pmatrix} 3 & 0 \\ 4 & 5 \\ 1 & 2 \end{pmatrix}$$

아래 예제를 통해 전치 행렬을 구해보라.

PYTHON

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

A_T = A.T

print("원본 행렬 A:\n", A)
print("전치 행렬 A^T:\n", A_T)
```

- 이외 특수한 행렬들

특수한 행렬

정사각행렬

- 행과 열의 개수가 같은 행렬이다.

$$\begin{pmatrix} 2 & 0 & 1 \\ 1 & 21 & 5 \\ 4 & 5 & 12 \end{pmatrix}$$

대각행렬

- 주대각선 이외의 원소가 모두 0인 행렬이다.

$$\begin{pmatrix} 50 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 8 \end{pmatrix}$$

단위행렬

- 주대각선의 원소가 모두 1이고, 나머지 원소가 모두 0인 행렬이다.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

대칭행렬

- 전치행렬이 자기 자신과 같은 행렬이다.

$$A = A^T$$
$$\begin{pmatrix} 1 & 2 & 11 \\ 2 & 21 & 5 \\ 11 & 5 & 1 \end{pmatrix}$$

Tensor

텐서는(tensor)는 수학적으로 **벡터, 행렬, 그 이상의 차원을 갖는 다차원 배열**을 나타내는 개념으로 다양한 형태의 데이터를 나타내는 데 사용되는 다차원 배열이다. 특히 기계 학습 분야 중 **deep learning 분야에서 데이터를 효율적으로 저장하고 연산**하는 데 사용된다.

즉 데이터의 배열이라 말할 수 있다.

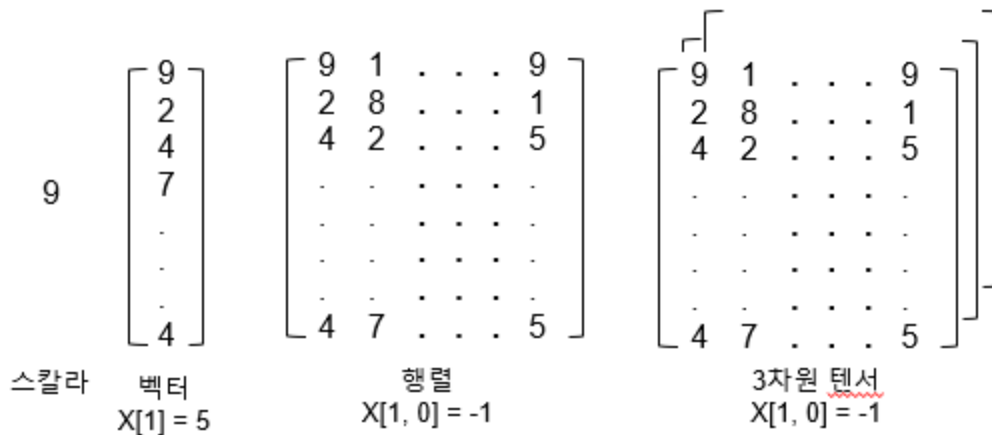
- 다차원 배열로 확장 및 연산: 숫자들로 구성된 데이터를 고려하면, 이 데이터를 **1차원 배열로 혹은 2차원 배열 혹은 3차원 배열**로 만들 수 있다. 쉽게 데이터를 확장 및 조작을 가능케 한 것이 텐서이다.
- 예를 들어 흑백이미지는 width와 height를 가진 2D 텐서(값은 흑백의 데이터 1byte), 컬러이미지는 width, height, color를 가진 3D 텐서(컬러가 R,G, B이면 3개의 색상 채널을 가짐. 즉, 각 색상 채널당 width와 height의 2D 데이터를 가짐)이다. 이러한 데이터들이 수학적 연산 및 텐서 연산을 통해 처리된다.
- 기계 학습에서의 활용: 텐서는 딥러닝에서 주요 데이터 구조이다. 그림 혹은 영상들은 텐서로 변환하여 학습 모델에 입력으로 제공하며, 학습 모델의 가중치(weight)와 bias도 텐서로 표현된다. 텐서는 딥러닝에서 데이터 및 학습모델의 핵심 구성 요소이다.

텐서의 RANK와 SHAPE

- Rank: 텐서의 차원수를 말한다.
- Shape: **shape**은 각 차원의 크기를 나타내는 튜플(tuple)이며, 텐서의 각 차원에 대한 크기를 순서대로 나타낸다. 즉, rank가 N이면 N-tuple로 구성된다.
 - 예를 들어 3차원 텐서의 shape가 (2, 3, 4)이면, 이는 첫 번째 차원의 길이가 2, 두 번째 차원의 길이가 3, 세 번째 차원의 길이가 4인 배열을 의미한다. 여기서 길이는 요소의 개수를 의

미한다. 따라서 **shape**은 텐서의 크기와 구조를 완전하게 설명해 주는 데이터이므로 shape를 알면 텐서가 어떤 구조를 가지고 있는지 파악할 수 있다.

차원	명칭	예시	핵심 사용법
0D	스칼라	<code>torch.tensor(5)</code>	손실 함수 값
1D	벡터	<code>torch.tensor([1, 2, 3])</code>	1개의 샘플의 특성값
2D	행렬	<code>shape: (H, W)</code>	흑백 이미지
3D	3차원 텐서	<code>shape: (C, H, W)</code>	컬러 이미지
4D	4차원 텐서	<code>shape: (N, C, H, W)</code>	배치 단위 컬러 이미지 입력



numpy를 활용하여 Rank와, shape, 그리고 기본적인 연산을 확인할 수 있다.

- `a.ndim`은 Numpy 배열(텐서)의 차원의 수(Rank)를 확인하는 속성으로, number of dimensions의 약자이다.
- `a.shape`는 각 차원의 크기를 나타내는 속성이다.
- `a.size`는 전체 원소의 개수를 나타내는 속성이다.
- `a.dtype`은 데이터의 자료형을 나타내는 속성이다.

```

import numpy as np

scalar = np.array(7)
print("스칼라: ", scalar)
print("Rank: ", scalar.ndim)
print("Shape: ", scalar.shape)

vecotr = np.array([1, 2, 3, 4, 5])
print("\n 벡터:", vecotr)
print("Rank: ", vecotr.ndim)
print("Shape: ", vecotr.shape)

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("\n 행렬: \n", matrix)
print("Rank: ", matrix.ndim)
print("Shape: ", matrix.shape)

tensord3d = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]],

    [[12, 14, 15, 16],
     [17, 18, 19, 20],
     [21, 22, 23, 24]]
])

print("\n 3차원 텐서: \n", tensord3d)
print("Rank: ", tensord3d.ndim)
print("Shape: ", tensord3d.shape)

```

텐서는 PyTorch나 TensorFlow를 이용해 딥러닝 모델을 구현할 때, 텐서가 매우 핵심적인 역할을 한다.

pandas

pandas는 Python에서 데이터를 쉽고 효율적으로 다룰 수 있도록 도와주는 오픈소스 데이터 분석 라이브러리이다. 이름은 "panel data"에서 유래되었으며, 주로 표 형태의 데이터(행과 열로 구성된 테이블)를 처리하는 데 최적화되어 있다. pandas는 특히 엑셀 처럼 데이터를 다룰 수 있으면서도, SQL처럼 필터링, 집계, 병합 등의 연산을 코드로 수행할 수 있는 점에서 데이터 과학자, 분석가, 엔지니어들에게 필수 도구로 여겨진다.

pandas의 핵심 객체는 **Series**와 **DataFrame**이다.

Series는 1차원 데이터(리스트나 배열과 유사)를 다루며, **DataFrame**은 여러 개의 **Series**가 행과 열 구조로 결합된 2차원 데이터 구조이다. pandas는 CSV, Excel, SQL, JSON등 다양한 파일을 읽고 쓸 수 있으며, 누락값 처리, 필터링, 그룹화, 통계 연산, 시계열 분석등 수많은 기능을 직관적인 문법으로 제공한다.

예를 들어 대용량의 엑셀 파일을 불러와 특정 조건에 따라 정리하거나, 웹에서 수집한 데이터를 통계적으로 요약하고 시각화 라이브러리로 넘겨주는 등 작업을 효율적으로 수행할 수 있다. 또한 pandas는 **NumPy**와 matplotlib과 같은 다른 과학 연산 도구들과 잘 호환 되어 **Python 기반 데이터 분석 파이프라인의 중심 역할을 한다.**

기능	엑셀	SQL	pandas
대용량 처리	X	O	O (메모리 제안 내)
복잡한 계산	X	O	O
인터랙티브 탐색	O	X	O (Jupyter/Colab)

Series(1차원)

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['A', 'B', 'C'])
print(s)
print(s.index)    # Index(['A', 'B', 'C'], dtype='object')
print(s.values)   # array([10, 20, 30])
```

PYTHON

Series는 **값(value)**과 **인덱스(index)**가 한 상으로 묶인 **1차원 배열**을 말한다.

넘파이 배열과 달리 **인덱스를 활용한 라벨 기반 접근**이 가능하다.

DataFrame(2차원)

```
import pandas as pd

data = {
    "이름": ["철수", "영희", "민수"],
    "국어": [90, 85, 78],
    "수학": [95, 88, 82]
}

df = pd.DataFrame(data)
print(df)
```

PYTHON

DataFrame은 **Series**가 같은 인덱스를 공유하며 모인 표 형태의 자료구조이다. 열(column)마다 자료형이 달라도 된다.

DataFrame은 내부적으로 **numpy 배열 기반**이기 때문에 **벡터 연산, 조건 연산, 브로드캐스팅**이 가능하다.

PYTHON

```
import pandas as pd
import pandas as pd

df = pd.DataFrame({
    "국어": [90, 85, 78],
    "수학": [95, 88, 82]
}, index=["철수", "영희", "민수"])

# 열 단위 연산: 평균 점수 계산 (각 행에 대해)
df["평균"] = (df["국어"] + df["수학"]) / 2

#브로드캐스팅: 수학 점수에 5점 가산
df["수학+보너스"] = df["수학"] + 5

print(df)
```

출력 예시

	국어	수학	평균	수학+보너스
철수	90	95	92.5	100
영희	85	88	86.5	93
민수	78	82	80.0	87

- **numpy**처럼 연산자를 그대로 사용해 계산이 가능하며, 열 단위로 자동 정렬됨.
- **df["평균"] = np.mean(df, axis=1)**처럼 **numpy** 함수도 그대로 적용이 가능하다.

아래 표는 **pandas의 핵심 기능과 대표적인 메서드**들을 정리한 것이다. 데이터프레임을 다룰 때 자주 사용되는 기능들이며, 데이터 탐색부터 전처리, 분석, 저장 가지 전 과정을 간결한 코드로 수행할 수 있다.

기능	메서드(예시)	설명
행/열 미리 보기	<code>df.head(3)</code> , <code>df.tail()</code>	상·하단 일부 출력
행·열 선택	<code>df['열']</code> , <code>df.loc[행라벨, '열']</code> , <code>df.iloc[행번호, 열번호]</code>	라벨/정수 기반 인덱싱
조건 필터링	<code>df[df['점수'] > 80]</code>	불린 마스크
정렬	<code>df.sort_values('점수', ascending=False)</code>	값 기준 정렬
통계 요약	<code>df.describe()</code> , <code>df.mean()</code>	기초 통계량
결측치 처리	<code>df.isna()</code> , <code>df.dropna()</code> , <code>df.fillna(값)</code>	NA 탐지/삭제/대체

기능	메서드(예시)	설명
그룹 분석	<code>df.groupby('반')['점수'].mean()</code>	그룹별 집계
데이터 병합	<code>pd.concat([df1, df2])</code> , <code>pd.merge(df1, df2, on='키')</code>	행·열 이어붙이기 / 조인
파일 입출력	<code>pd.read_csv()</code> , <code>df.to_excel()</code>	CSV, Excel, JSON 등

예시 코드들

PYTHON

```
import pandas as pd

# 샘플 데이터 생성 (학생 이름, 반, 국어, 수학 점수 포함)
data = {
    "이름": ["철수", "영희", "민수", "지민", "현우"],
    "반": ["A", "B", "A", "B", "A"],
    "국어": [85, 90, 78, 92, 88],
    "수학": [80, 95, 75, 89, 84]
}

# 데이터프레임 생성
df = pd.DataFrame(data)

# 평균 점수 컬럼 추가
df["평균"] = (df["국어"] + df["수학"]) / 2

# 출력
print("데이터프레임 예시:\n", df)
```

- `df.head()`와 `df.tail()`은 데이터의 상위 또는 하위 일부 행을 미리 보는 데 사용된다.

PYTHON

```
# 1. 행·열 미리 보기
print("\n✅ 상위 3개 행 미리 보기:")
print(df.head(3))

print("\n✅ 하위 2개 행 미리 보기:")
print(df.tail(2))
```

- `df['열']`, `df.loc[]`, `df.iloc[]`은 열 또는 행을 선택할 때 사용되며, 라벨이나 숫자 인덱스를 기준으로 접근할 수 있다.

2. 행·열 선택

```
print("\n✅ '수학' 열만 보기:")
print(df["수학"])

print("\n✅ 첫 번째 행, '국어' 점수 보기 (iloc):")
print(df.iloc[0, 2]) # 0번째 행, 2번째 열 (국어)

print("\n✅ '지민'의 평균 점수 보기 (loc):")
print(df.loc[df["이름"] == "지민", "평균"])
```

- `df[df['점수'] > 80]` 처럼 조건식을 활용하면 원하는 조건에 맞는 데이터만 필터링할 수 있다.

3. 조건 필터링

```
print("\n✅ 평균 점수가 85 이상인 학생:")
print(df[df["평균"] >= 85])
```

- 정렬은 `df.sort_values()` 를 사용하며, 특정 열 기준으로 오름차순 또는 내림차순 정렬이 가능하다.

✅ 4. 정렬

```
print("\n✅ 수학 점수 기준 내림차순 정렬:")
print(df.sort_values("수학", ascending=False))
```

- `df.describe()` 는 기초 통계 정보를 요약해주고, `df.mean()` 은 평균값을 계산하는 함수이다.

5. 통계 요약

```
print("\n✅ describe() 통계 요약:")
print(df.describe())

print("\n✅ 평균 점수만 평균값 구하기:")
print(df["평균"].mean())
```

- 결측치 처리는 `df.isna()`, `df.dropna()`, `df.fillna()` 등을 통해 수행하며, NA 값을 탐지하거나 삭제하거나 다른 값으로 대체할 수 있다.

✅ 6. 결측치 처리 (예시용 - 실제로 결측치는 없음)

```
print("\n✅ 결측치 탐지:")
print(df.isna())
```

- 그룹 분석에서는 `df.groupby()` 를 사용하여 특정 열 기준으로 데이터를 집계하거나 평균을 낼 수 있다.

```
# ✅ 7. 그룹 분석 (반별 평균)
print("\n✅ 반별 평균 점수:")
print(df.groupby("반")["평균"].mean())
```

- 데이터 병합은 `pd.concat()` 을 통해 행이나 열 방향으로 이어붙일 수 있고, `pd.merge()` 로 SQL처럼 조인 연산을 할 수 있다.

```
# 8. 데이터 병합
print("\n✅ 다른 데이터프레임과 이어붙이기 예제:")
df2 = pd.DataFrame({
    "이름": ["정우"],
    "반": ["B"],
    "국어": [91],
    "수학": [87]
})

df2["평균"] = (df2["국어"] + df2["수학"]) / 2

df_combined = pd.concat([df, df2], ignore_index=True)
print(df_combined)
```

- `pd.read_csv()`, `df.to_excel()` 등은 외부 파일과 연동하여 데이터를 읽거나 저장할 때 사용된다.

```
# ✅ 9. 파일 입출력
print("\n✅ CSV 파일로 저장 및 불러오기:")

df.to_csv("score.csv", index=False)

df_loaded = pd.read_csv("score.csv")
print("불러온 데이터프레임:\n", df_loaded)
```

이런 기능을 통해 pandas는 복잡한 데이터 처리 과정을 단순화 하고, 효율적인 분석 환경을 제공한다.

아래 표는 pandas를 이용한 외부 파일의 입출력 방법을 정리한 것이다.

pandas는 CSV, Excel, JSON 등 다양한 형식의 데이터를 읽고 쓰는 기능을 제공하며, 아래와같이 간단한 함수 호출로 이를 처리할 수 있다.

형식	읽기	쓰기
CSV	<code>pd.read_csv('data.csv')</code>	<code>df.to_csv('out.csv', index=False)</code>
Excel	<code>pd.read_excel('data.xlsx')</code>	<code>df.to_excel('out.xlsx', index=False)</code>
JSON	<code>pd.read_json('data.json')</code>	<code>df.to_json('out.json')</code>

- **CSV 파일**은 `pd.read_csv()`로 읽고, `df.to_csv()`로 쓸 수 있다. 저장 시 `index=False` 옵션을 주면 인덱스 열 없이 저장된다.
- **Excel 파일**은 `pd.read_excel()`로 읽으며, 저장은 `df.to_excel()`을 사용한다. 역시 `index=False` 옵션을 통해 인덱스를 제외할 수 있다.
- **JSON 파일**은 `pd.read_json()`으로 읽고, `df.to_json()`으로 저장한다.

이처럼 pandas는 파일 형식에 관계없이 일관된 방식으로 데이터를 불러오고 저장할 수 있도록 도와주며, 데이터 분석 워크플로우에서 입출력을 손쉽게 처리할 수 있게 해준다.

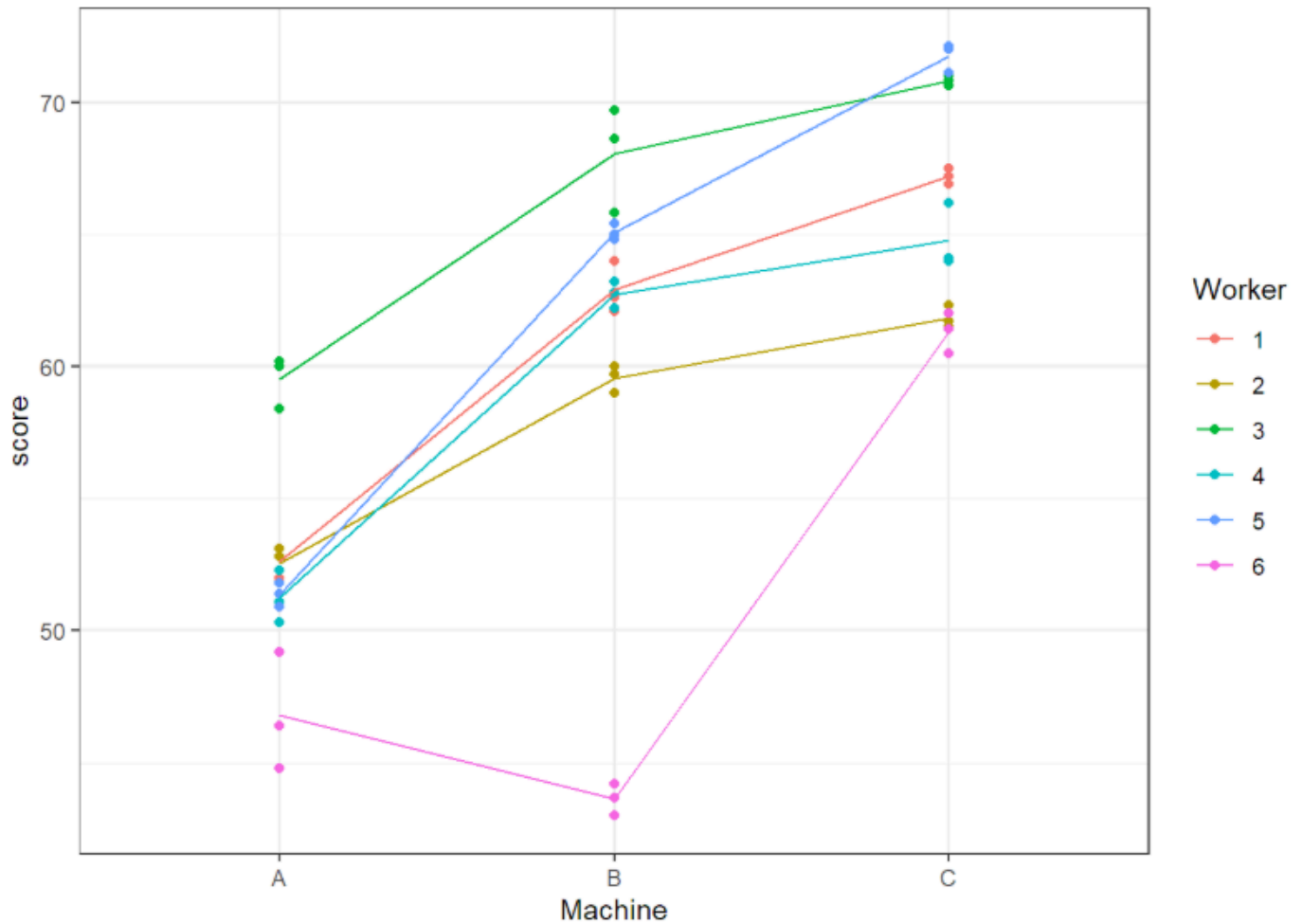
Matplotlib

Matplotlib은 2D 그래프를 그릴 때 가장 많이 사용되는 Python 라이브러리 중 하나이다. 해당 라이브러리를 이용하여 데이터를 다양한 형태로 visualization할 수 있고 논문에 기재될 만큼 좋은 표현될 수 있다.

Pyplot

Pyplot은 MATLAB 언어에서 유래된 모듈로, MATLAB에서 구사하는 plotting과 유사하게 Python을 이용하여 사용할 수 있게 해준다.

기본적으로 머신러닝과 딥러닝에서는 시각화가 굉장히 중요하다. 모델을 훈련시키기전 데이터 전처리 단계에서도 모델을 훈련시키고 난 뒤에도 시각화를 통해 알 수 있다.



기본 그래프 그리기

`pyplot.plot()` 함수에 하나의 숫자 리스트를 입력함으로써 그래프가 그려진다.

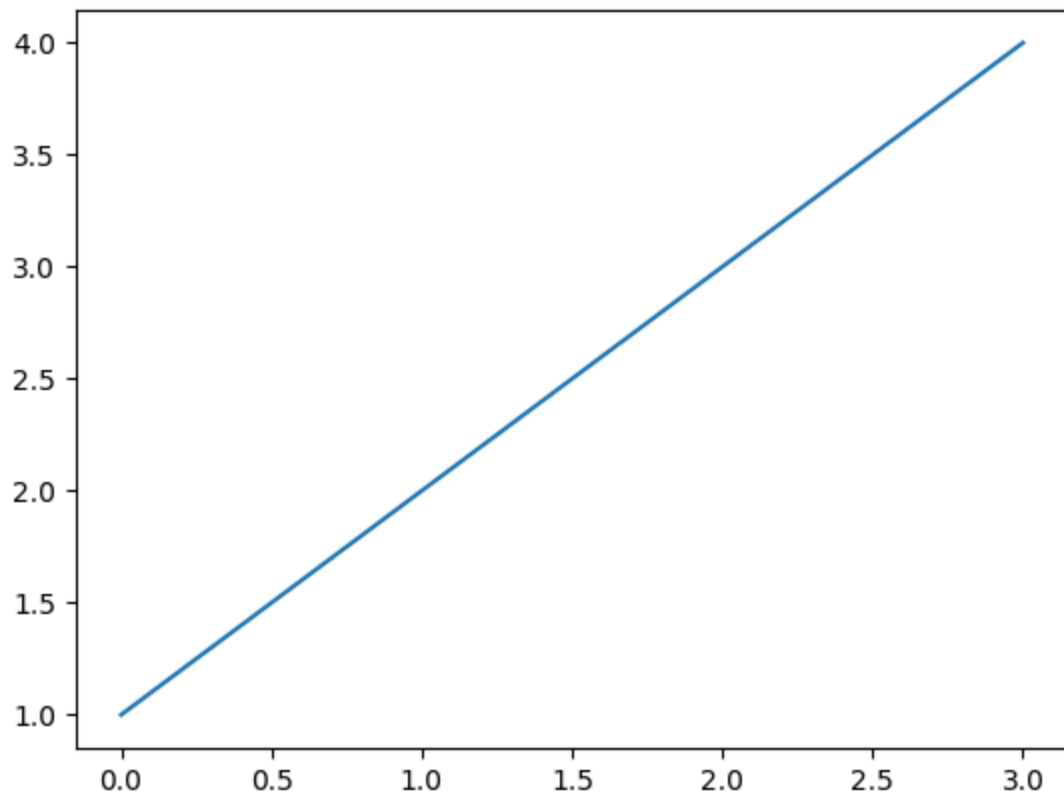
```
import matplotlib.pyplot as plt
```

```
plt.plot([1, 2, 3, 4])
```

```
plt.show()
```

PYTHON

- `plot()` 함수는 리스트의 값들이 y 값들이라고 가정하고, x 값 `[0, 1, 2, 3]`을 자동으로 만들어 낸다.
- `show()` 함수는 그래프를 화면에 나타나도록 한다.



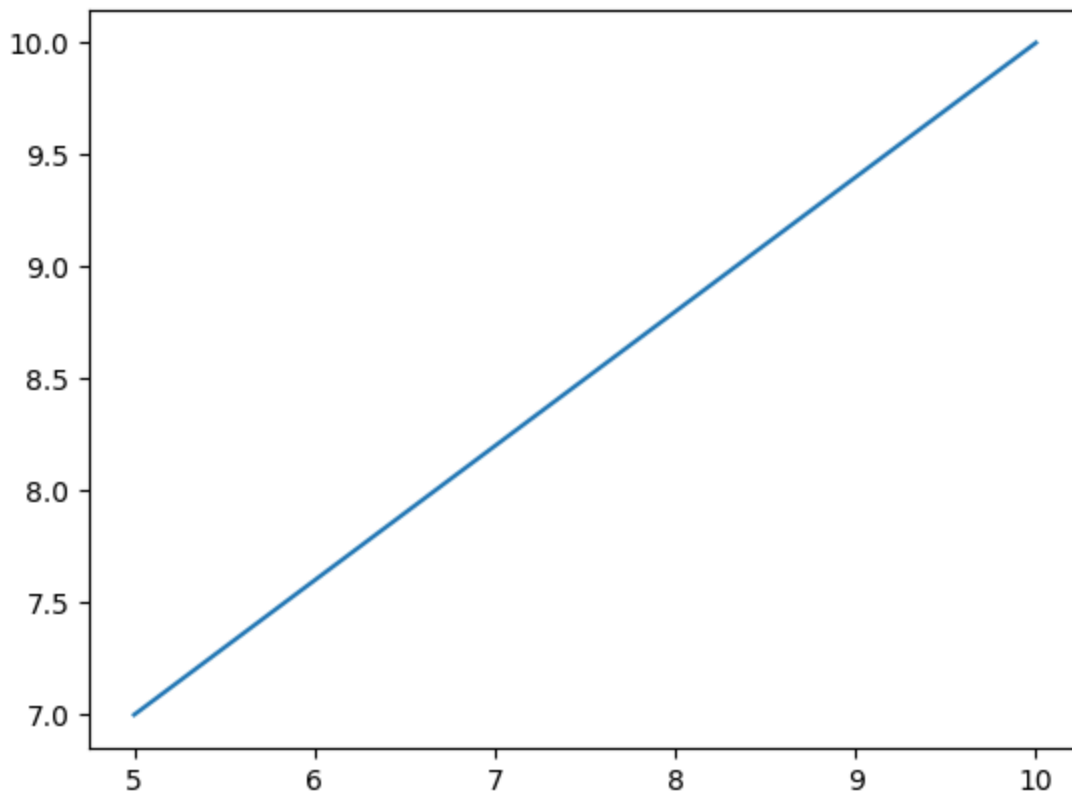
`numpy`와 연계하여 사용도 가능하다.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([5, 10])
ypoints = np.array([7, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



x축과 y축은 `plot(x축, y축)`으로 지정 가능하다. 또한 동시에 `array([5, 10], [7, 10])`와 같은 방식으로 지정이 가능하다.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

points = np.array([5, 10], [7, 10])

plt.plot(points)
plt.show()
```

Markers

argument `marker`을 이용해서 각각의 점들을 다르게 표시하여 강조 할 수 있다.

- `marker` = '모양'
- `fmt` = 'marker/line/color'
- `ms` : markersize
- `mec` : markeredgecolor
- `mfc` : markerfacecolor

`marker = 'o'`으로 circle로 점 d를 강조한다.

```
# point [(0,3), (1,7), (2,2), (3,10)]
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 1, 2, 3])
ypoints = np.array([3, 7, 2, 10])

plt.plot(ypoints, marker = 'o')
plt.show()
```

Scatter Plot 산점도 그래프 그리기

scatter plot은 데이터를 관찰 할 때, 전체적인 분포도를 보기 위하여 각 개체를 점으로 표시하여 나타낸 그래프이다. 여러개의 분포를 볼 수 있다. 여러 변수의 상관 관계를 파악 하기 위해 사용된다.

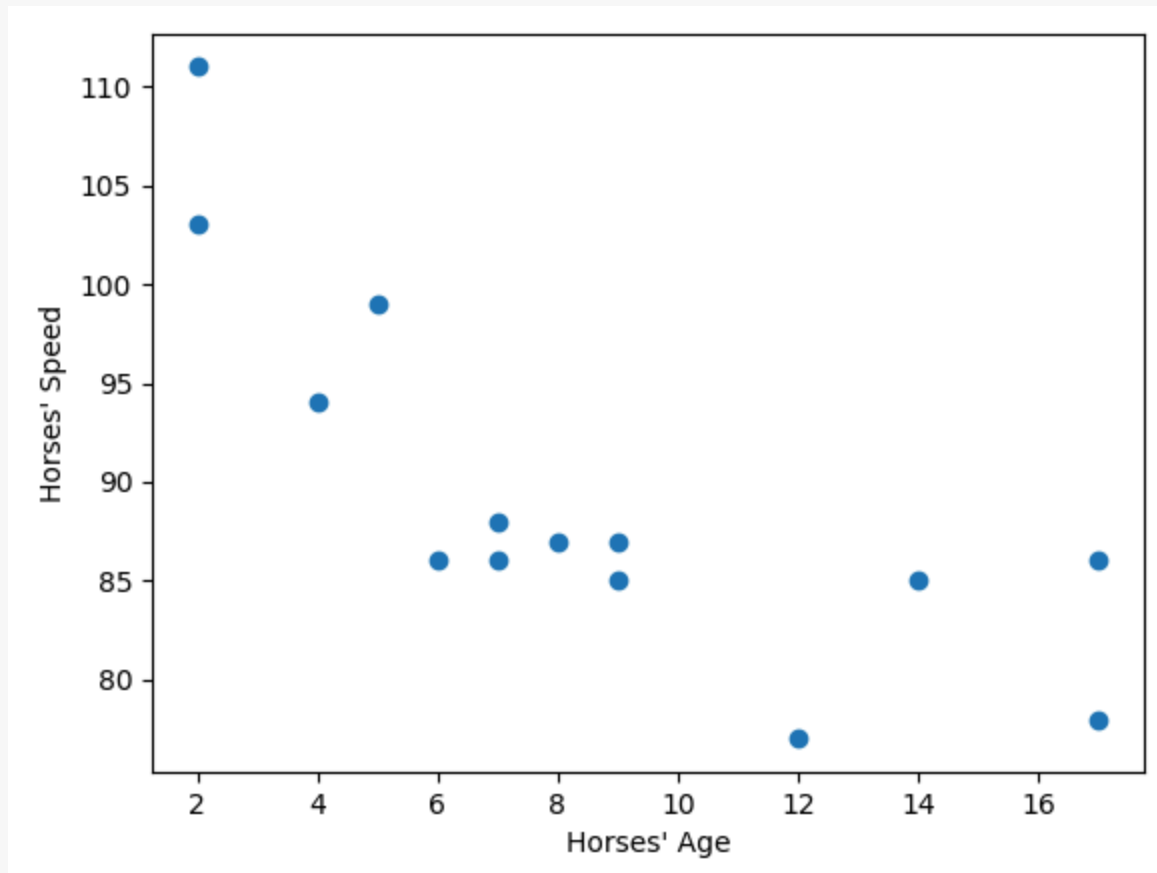
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,17,12,9,6,14])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86,85])

plt.xlabel("Horses' Age")
plt.ylabel("Horses' Speed")

plt.scatter(x, y)
plt.show()
```

출력 결과



여러 데이터를 더 얻어서 합치고 싶으면 `plt.scatter()`를 여러개 해주면 된다.

또한 Scatter plot의 dot(점) 색을 변경하고 싶으면 `plt.scatter( x, y, color = '□color code')`에서 '□color code'을 원하는 색으로 변경하면 된다.

```

import matplotlib.pyplot as plt
import numpy as np

plt.xlabel("Horses' Age")
plt.ylabel("Horses' Speed")

#day one, the age and speed of 14 horses:
x = np.array([5,7,8,7,2,17,2,9,4,17,12,9,6,14])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86,85])
plt.scatter(x, y, color = 'hotpink')

#day two, the age and speed of 16 horses:
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12,15])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85,90])
plt.scatter(x, y, color = '#88c999')

plt.show()

```

출력 결과

